Runbook — deploy, rollback and incident response

> Milestone 3 evidence: "*Ops runbook (deploy/rollback/incident) in repo*". Covers `apps/web` and `apps/api` on the **Render free tier**, with the database on **Turso**. The infrastructure artifact is `render.yaml` at the repository root, and it is the single source of truth for configuration: this document explains **how to operate it**, not what the YAML says.
>
> **Everything runs at $0/month.** That imposes non-negotiable constraints — see §Accepted limitations.

0 · What gets deployed

Service	What it is	URL	Starts with
`propnexus-api`	Express 5 on Node	`https://propnexus-api.onrender.com`	migrations + `server.js`
`propnexus-web`	Static SPA (TanStack Router + Vite)	`https://propnexus-web.onrender.com`	served from a CDN, no process
database	Managed SQLite	`libsql://…turso.io`	—

The web app proxies nothing. As a **static SPA** it lives on another origin and calls the API's absolute URL, so:

- the browser **does** see the API URL, and there **is** a CORS preflight;
- the API accepts the web app through an **allowlist** (`WEB_ORIGIN`), never with `*`;
- `VITE_API_ORIGIN` is **build time** — changing it requires a *redeploy* of the web app, not a restart;
- `WEB_ORIGIN` is **runtime** on the API: it takes effect with a restart.

Those two variables are the pair that makes the app work. If one is missing, the web app loads and no request goes through. See §2.

1 · First-time setup

Done once. Requires Render and Turso accounts (both free, no card).

1.1 · Database on Turso

```
turso auth login                                          # opens the browser
turso group create propnexus --location aws-us-west-2     # Render's region (Oregon)
turso db create propnexus-west --group propnexus          # free: 5 GB · 500M reads · 10M writes/month
turso db show propnexus-west --url                        # → libsql://propnexus-west-<org>.aws-us-west-2.turso.io
turso db tokens create propnexus-west                     # → the DATABASE_AUTH_TOKEN
```

The database goes in the API's region. With Render in Oregon and the database in Virginia, each round trip to the database cost ~70–120 ms; in the same region, ~16 ms. A database cannot move between groups: changing region means a new database loaded from a `.dump`.

Keep both values: they go into the Render dashboard, **never into the repository**.

1.2 · The two services on Render

```
render login
```

In the dashboard: **New** → **Blueprint**, choose the repository `Javote/real-world-real-estate`, branch `main`. Render reads `render.yaml` and proposes both services — **the API as a Node service and the web app as a static site**. It asks for the values marked `sync: false`:

Service	Variable	Value
`propnexus-api`	`DATABASE_URL`	the `libsql://…` from 1.1
`propnexus-api`	`DATABASE_AUTH_TOKEN`	the token from 1.1
`propnexus-api`	`WEB_ORIGIN`	**the web app's URL, with scheme** — the CORS allowlist
`propnexus-web`	`VITE_API_ORIGIN`	**the API's URL, with scheme**

`JWT_SECRET` **is not requested**: Render generates it (`generateValue: true`). Do not set it by hand and do not copy it from a local `.env`.

The two URLs cross-reference each other, and that has an order. The web app needs to know where the API is **at build time**; the API needs to know where the web app is for CORS, but that is runtime and can be set later. Safe procedure:

1. Let **the API** deploy first and copy its URL from the dashboard.
2. Paste it into the web app's `VITE_API_ORIGIN` and trigger **Manual Deploy → Clear build cache & deploy**.
3. Copy the **web app's** URL and paste it into the API's `WEB_ORIGIN`. That one takes effect with a restart.

If Render had to add a suffix to a hostname because the name was taken, the real URL is not the one in the table: always use the one the dashboard shows.

Symptom of skipping step 3: the app loads, login does not respond, and the browser console shows a CORS error. The API is not down — its allowlist does not include that origin.

1.3 · Seeding the demo accounts

The free tier **has no remote shell**, so the seed runs from your machine against Turso. **It is `db:seed:produccion`, not `db:seed`**: the local one uses the passwords published in the repository and refuses to run against anything but a SQLite file on disk.

The seed does NOT print passwords that came from the environment — it prints `(from SEED_ADMIN_PASSWORD)`. This is deliberate, pinned by a test (`apps/api/test/seed-demo.test.ts`), and it is why this procedure has **two steps**: if you generate the passwords inline with `$(openssl …)`, nobody ever sees them and you end up with five accounts whose credentials nobody knows — with no remote shell to fix it.

Generate them and **store them first**:

```
ADMIN_PW=$(openssl rand -base64 24)
DEMO_PW=$(openssl rand -base64 24)
echo "admin@example.com  → $ADMIN_PW"    # ← copy them to the password manager NOW
echo "the other four     → $DEMO_PW"
```

Only then, **in that same shell** (the variables only exist there):

```
DATABASE_URL='libsql://propnexus-west-<org>.aws-us-west-2.turso.io' \
DATABASE_AUTH_TOKEN="$(turso db tokens create propnexus-west)" \
SEED_ADMIN_PASSWORD="$ADMIN_PW" \
SEED_DEMO_PASSWORD="$DEMO_PW" \
JWT_SECRET=anything-the-seed-signs-nothing \
pnpm --filter @plataforma/api db:seed:produccion
```

`$(turso db tokens create propnexus-west)` avoids copying and pasting the token. Tokens are **additive**: creating a new one does not invalidate the one already in Render. What does break the deployed API is `turso db tokens invalidate`, which kills **all of them** at once — do not run it.

The passwords are mandatory and `db:seed:produccion` fails without them, before writing anything: it has no defaults. The local seed's credentials are published in the repository, and seeding them into a deployed instance would leave an admin account with known credentials. The ones you stored in the first step are the ones you hand to a reviewer.

Migrations do **not** need to be run by hand: they are part of the API's `startCommand` and are idempotent (`_migrations` table).

1.4 · The evidence bucket on Cloudflare R2

R2 free tier: **10 GB of storage and $0 egress** — the latter is why R2 and not S3.

The driver already exists and is the same one tested against MinIO locally: no code is written here, you create a bucket and a key pair.

1. Cloudflare dashboard → **R2** → *Create bucket* → name **`propnexus-evidencia`** (it must match `S3_BUCKET` in `render.yaml`). Location *Automatic*.

2. **R2** → **Manage API Tokens** → **Create API Token**, of type **Account**, permission **Object Read & Write**, scoped to that bucket.

   Account and not User, and it matters. A user token "*inherits your personal permissions and becomes inactive if your user is removed from the account*": it would tie the persistence of all evidence to that person keeping their access. An account token

"*remains valid until manually revoked*". And **Object** Read & Write, not *Admin*: the API neither creates nor deletes buckets, hence `S3_CREATE_BUCKET=false`.

Cloudflare shows the pair **only once**: `Access Key ID` + `Secret Access Key`.

3. Note the **endpoint**. Cloudflare gives the *jurisdiction-specific endpoint* ready-made: **use it verbatim**. If the bucket ended up in a jurisdiction (EU, for example) the URL has that segment in the middle, and the generic form `https://<ACCOUNT_ID>.r2.cloudflarestorage.com` **does not work**.

On the same screen Cloudflare also hands out a **token for its REST API**. That one does **not** go to Render: the driver speaks S3 and authenticates with the key pair. Loading it would be one more secret in the environment, with more permissions than needed and nothing reading it.

Test against R2 BEFORE deploying. The same integration test that runs against MinIO works when pointed at R2, and it is the only way to know the credentials work without risking the API:

```
S3_TEST=1 \
S3_ENDPOINT='https://<ACCOUNT_ID>.r2.cloudflarestorage.com' \
S3_BUCKET=propnexus-evidencia \
S3_ACCESS_KEY_ID='<access key>' \
S3_SECRET_ACCESS_KEY='<secret>' \
S3_REGION=auto S3_FORCE_PATH_STYLE=false S3_CREATE_BUCKET=false \
pnpm --filter @plataforma/api exec vitest run test/storage-s3.test.ts
```

The last two variables **are not redundant**: the code defaults target MinIO (`us-east-1` and path-style) and R2 wants `auto` and virtual-hosted style.

⚠ **The order is mandatory, not a recommendation.** `STORAGE_DRIVER=s3` without the `S3_*` variables makes the API **refuse to start** — on purpose: it fails at startup rather than on the first upload, with a user waiting. So the credentials go into the Render dashboard **before** the `render.yaml` with `STORAGE_DRIVER=s3` reaches `main`. If the order is reversed, the API stays down until they are loaded.

When syncing the Blueprint, Render asks for the three new `sync: false` values: `S3_ENDPOINT`, `S3_ACCESS_KEY_ID` and `S3_SECRET_ACCESS_KEY`.

1.5 · Real anchoring on Preprod

Optional: the instance works with `ANCHOR_MODE=simulated`, the default. This switches it to anchor for real on Cardano Preprod.

1. **Blockfrost** — free account, **Preprod** project, copy the key (it starts with `preprod`).
2. **The service wallet**, with the repository's generator:

   ```
   BLOCKFROST_API_KEY='<preprod key>' pnpm --filter @plataforma/cardano wallet:new
   ```

   It writes the **payment key** to `~/propnexus-wallet-preprod.key` with `600` permissions and **does not print it**; it prints the address and the admin, which are public. It refuses to overwrite an existing file.

   It is a payment key and not a seed phrase: **one key, once**. The address is derived from it, so there is no second variable that could fall out of sync.

   ⚠ **It is not rotated.** The validator's admin is the hash of this key: replacing it leaves the already-anchored threads unreachable, with no visible error.

3. **Fund the address the generator printed** from the faucet — **Preprod**, not Preview. An anchor costs ~0.17 tADA and the reference script of step 6 locks ~11 tADA, so it is more than enough.

   The faucet asks for an address, not a key. It must be exactly the one the service uses; the API prints it again at startup (`Wallet de servicio: addr_test1…`) so it can be compared without deriving anything by hand.

   Check the balance without the key, against Koios:

   ```
   curl -s -X POST https://preprod.koios.rest/api/v1/address_info \
     -H 'Content-Type: application/json' -d '{"_addresses":["addr_test1..."]}'
   ```

4. **Test locally before touching the deployment**, the same order used for R2:

```
export BLOCKFROST_API_KEY='<key>'
export SERVICE_WALLET_PRIVATE_KEY="$(cat ~/propnexus-wallet-preprod.key)"
DATABASE_URL='file:./apps/api/.data/dev.db' JWT_SECRET=local \
ANCHOR_MODE=real CARDANO_NETWORK=Preprod PORT=8788 \
node apps/api/dist/src/server.js
```

The line to look for is `AnchorPort listo en modo "real"` ("AnchorPort ready in real mode"). If it says `simulated`, the variable did not arrive; if the process died, the error says what is missing.

5. **On Render**, on `propnexus-api`: `BLOCKFROST_API_KEY` and `SERVICE_WALLET_PRIVATE_KEY`. They are runtime and take effect with a restart. `ANCHOR_MODE` **is no longer set by hand**: the Blueprint declares it with `value: real`, so a dashboard change would be overwritten by the next re-sync.

   If either one is missing, the API still starts with anchoring disabled. The startup says so — `AnchorPort listo en modo "disabled"`, with the reason on the line above — and every anchor is rejected, so it never writes a TXID that does not exist. It is the safe state, not the good state: until they are loaded, the platform records but does not prove.

6. **Publish the validator as a reference script.** Done **once per network**, as soon as the wallet is funded, independent of the previous steps:

   ```
   BLOCKFROST_API_KEY='<key>' \
   SERVICE_WALLET_PRIVATE_KEY="$(cat ~/propnexus-wallet-preprod.key)" \
   pnpm --filter @plataforma/cardano ref:publish
   ```

   It leaves a UTxO holding the validator at the wallet's address. From then on, every thread transaction **references** it instead of carrying it: from 2,890 to 599 bytes, from 0.2976 to 0.2317 tADA. **It is idempotent**: running it twice does not publish twice or spend again.

   It locks ~11 tADA — the minimum Cardano requires for a UTxO holding a script, not value. It stays in the wallet and can be recovered with a deliberate transaction.

   ⚠ **Restart the API afterwards.** The reference script is discovered when the process starts: an instance already running keeps attaching the validator until the next deploy or restart. It breaks nothing; it just overpays.

Verify an anchor against the chain, without the key, with the TXID the endpoint returns:

```
curl -s -X POST https://preprod.koios.rest/api/v1/tx_metadata \
  -H 'Content-Type: application/json' -d '{"_tx_hashes":["<txid>"]}'
```

The metadata carries label `1904` with `h` (the SHA-256) and `r` (an **opaque** reference, never the file name or personal data).

An anchor is born `Pending` and has to be reconciled. Anchoring leaves the event `Pending` because at that moment the transaction has been *submitted*, not confirmed, and the platform never claims a proof it cannot substantiate. What promotes it to `Confirmed` is:

```
curl -s -X POST https://propnexus-api.onrender.com/api/v1/evidence/reconcile \
  -H "Authorization: Bearer <admin token>"      # → {"revisados":N,"confirmados":M}
```

It queries `/txs/{hash}` on Blockfrost for each pending event with a TXID and confirms those already in a block. It is idempotent: triggering it too often costs nothing, and an event without a TXID is left alone — there is nothing to query.

It has to be triggered from outside: a `setInterval` inside the API stops counting when Render puts the service to sleep after 15 minutes, and the free tier has no workers. A GitHub Actions cron calls that same endpoint (`.github/workflows/reconcile.yml`, every day at 06:00 UTC) and fails if it finds suspicious threads. It can also be run by hand, and every screen that shows an anchor reconciles its own scope on read.

On Preprod a block takes ~20 s, so reconciling right after anchoring usually returns `confirmados: 0`. It is not an error: it has not confirmed yet. Run it again.

2 · Everyday deploy

Push to `main`. **Render deploys only if every GitHub check on the commit passes** (`autoDeployTrigger: checksPass` on both services, since 2026-10-07; pinned by `apps/api/test/render-config.test.ts`). Per Render's docs, a check passes if it concludes

`success`, `neutral` or `skipped`; **if any check fails, or the commit has none, there is no deploy**. Until that day both services were on `commit` and deployed without looking at CI: the six `SPEC-614` commits of 2026-10-06 reached production with CI red.

What this means in practice:

- **The deploy starts when CI finishes**, not on push: about 5 minutes later, the slowest job.
- **Red CI = nothing deploys.** The fix is a new commit; the deploy ships with it, and it includes the earlier ones, **but only if that commit touches something the `buildFilter` includes and does not ignore**. Render decides from the files in that push, not from what is still undeployed: on 2026-10-21, `51a26d6` (four specs) went red, the fix `95e4ea4` touched only a test and a `vitest.config.mts` —both in `ignoredPaths` of both services—, CI passed and neither service deployed. If the fix falls entirely in `ignoredPaths`, another commit touching an included path unblocks the deploy (it shipped with `render.yaml`).
- **A commit cancelled by `cancel-in-progress`** (two pushes in a row) does not deploy on its own: it ships with the next one.
- **A commit CI skips** (only `.md`, images, `.pdf`…) has no checks and does not deploy, even if the `buildFilter` includes it (an image under `apps/web/public/`): it ships with the next code commit.
- **The rollback in §3 and the dashboard's Manual Deploy do not wait for CI**: they are manual, on purpose.
- **How to check it is still on:** `render services list -o json` shows `"autoDeployTrigger": "checksPass"` on both. The Blueprint does not always apply or remove a field on its own (it happened with `NODE_ENV`, see `render.yaml`).

Until 2026-09-22, Render rebuilt **both services on every push**, whatever it touched.

The `buildFilter`s were declared and did not filter. Measured on 2026-08-27: commit `ee357df` touched only three documentation files — no `render.yaml` involved — and triggered new deploys of both services, with `trigger = new_commit`, exactly the case where they should apply. Measured again on 2026-09-22, over the API's last 100 deploys: **57 came from commits the filter excludes** (38 documentation-only, 17 `apps/web`-only).

The cause, fixed on 2026-09-22: the `rootDir: .` both services had. Render's docs say that with a root directory "*Render only triggers an autodeploy if your changes affect files anywhere under that directory*", and with `.` the whole repository falls inside it. The `rootDir` was removed and `ignoredPaths` were added (documentation, tests, test configs). **It worked:** on 2026-10-05, still on `autoDeployTrigger: commit`, `43055c0` (only `apps/web`) and `64abcc1` (only `.github/`) did not deploy the API. Watch out: a field removed from the Blueprint may not be removed from the service (it happened with `NODE_ENV`, see `render.yaml`).

The web app installed twice. On a static site, Render runs its own `pnpm install` of the whole workspace before the `buildCommand` (12.8 s of the ~33 s deploy), so the `--filter` saved nothing and the cache filled up again with the API's dependencies. Since 2026-09-22 the web app declares `SKIP_INSTALL_DEPS=true` and the only install is the `buildCommand`'s. The API does not need it: on a web service Render does not install on its own.

The deploy that coincides with the inactivity shutdown — 2026-09-22

Every failed API deploy of the preceding weeks had the same cause, and it was not the code. On the free plan, Render puts the service to sleep after **15 minutes without any HTTP request**, and doing so sends `SIGTERM` to **all** its instances, including the one being deployed. If a deploy starts 13-15 minutes after the last request, the shutdown lands in the middle of its startup: the new instance dies, Render keeps scanning for a port that no longer exists until its 15-minute timeout, the deploy ends `update_failed`, and **the API stays down for ~18 minutes**.

Verified on three of the five `update_failed` among the last 100 deploys, reading the logs with instance and type (`render logs … --output json`): between the previous `Your service is live` and the `SIGTERM` there are **zero requests and exactly 15:00 minutes**, all three times (`b10afdf`: 19:45:51 → 20:00:51; `d69d059`: 16:57:45 → 17:12:45; `9054681`: 17:34:50 → 17:49:50). The "slow" deploys (>400 s) are not a different problem: they are the ones queued behind one of these.

It does not depend on the kind of commit. That documentation-only commits failed more often was a timing coincidence: the documentation commit tends to arrive a while after the code commit, right in the 13-15 minute window. Of the five failures, one was an API code commit.

How to avoid it, by cost:

1. **A request to the API** (`curl -s https://propnexus-api.onrender.com/health`) when CI finishes: it resets the 15-minute counter, and the deploy (~2.5 min) finishes with room to spare.
2. **Deploy less**: the `buildFilter` above.
3. **A periodic ping** (every <15 min) that keeps the service awake. It would also remove the cold start of the first visit. **Owner's decision:** the free plan gives 750 hours per month **per workspace**, shared with other free services. One service awake all month is ~720 h: if another one also consumes hours, the workspace runs out and Render suspends them until the end of the month.

A request during the hang unstuck it, once. On 2026-10-07, the deploy of `fb81ba9` started 13 minutes after the last request — a production check —: `SIGTERM` to the old instance (16:58:44) and to the new one (16:58:46), and nothing else in the logs. At 17:07 a `curl` to `/health` started a fresh instance, and at 17:08:24 Render declared the deploy `live`, with no redeploy. Seen only once: if it does not happen, there is the `update_failed` and the redeploy.

This corrects two earlier diagnoses, which blamed the same symptom on something else: the 2026-09-04 incident ("*it started fine fourteen minutes earlier*") and the 2026-09-22 one (blamed on "*a slow Blockfrost*"). Both are this pattern.

"No open HTTP ports detected on 0.0.0.0, continuing to scan…"

It shows up in deploys while Render looks for the port, and it is not an error: on 2026-09-22, ~10 s after `API listening`; on 2026-10-07, 13 s before. The API opens the port before initializing the `AnchorPort` (on purpose, since 2026-09-22). But initializing it in `real` mode imports Lucid, which **blocks the event loop**: measured locally, a single 2.1 s block on a 12-core machine, and on the free plan's 0.1 CPU that is ~38 s. While it lasts, the process does not respond. In the three startups of 2026-10-07, `AnchorPort listo` arrived 48-60 s after `API listening`, and Render declared `live` before that (in the only deploy, 33 s after `API listening` and 15 s before `AnchorPort listo`).

What it costs: build minutes and a cold restart per commit. It no longer costs evidence — since R2, a redeploy takes nothing with it (§1.4). Before assuming a documentation commit is free, check: it is not.

A push that arrives while a service is suspended is lost, and resuming does not fully recover it. Measured on 2026-08-31: with both services suspended by hand, `c435274` was pushed; on resume, the web app was on the new commit and **the API stayed four commits behind**, both `live` and with no errors in the logs. The status did not show it: you had to look at the commit.

The asymmetry is the danger, not the lag. Production ended up with the new front end talking to the old API, and since the schemas in `packages/shared` are `z.strictObject`, a missing field breaks the whole parse: a screen that used to work stopped working without Render saying anything.

It is the flip side of the `buildFilter`: we already knew Render sometimes deploys **too much**, and now we know it sometimes deploys **too little**. Both lead to the same rule.

> **After resuming a service, check each one's commit — not that they say `live`.**
>
> ```
> render deploys list <srv-id> --output json --confirm < /dev/null | \
>   python3 -c 'import sys,json; y=json.load(sys.stdin)[0]; print(y["status"], (y.get("commit") or
>         {}).get("id","")[:7])'
> ```
>
> If one fell behind, the fix is a redeploy and nothing else — there is no state to repair: `render deploys create <srv-id> --output json --confirm`. It builds from `main`, and if the build fails Render keeps the current version alive, so there is no downtime window.
>
> The `< /dev/null` is not decoration: without it, the CLI swallows stdin and inside a `for` loop returns nothing.

GitHub Actions does not deploy. CI is the only judge of whether a change can be pushed; Render only reacts to what already went through it.

A new `sync: false` variable added to an existing service does NOT show up on its own in the dashboard. Render "asks" for the values in the interactive **New → Blueprint** wizard (first-time setup) and in a manual **Sync** from the Blueprints tab. A normal push to `main` that adds a new `sync: false` to an already-deployed service does **not** trigger that dialog: the variable is not created, neither empty nor with any value — confirmed on 2026-09-07. Go to **the service → Environment → Add Environment Variable** and create it by hand, with the exact `key` declared in `render.yaml`. Once created that way, Render treats it like any other `sync: false`: a future sync does not overwrite it.

Post-deploy verification, in this order:

```
curl -s https://propnexus-api.onrender.com/health                # {"ok":true}
curl -s -o /dev/null -w '%{http_code}\n' https://propnexus-web.onrender.com/   # 200, the SPA index

# Login goes DIRECTLY to the API (there is no proxy). With the web app's `Origin`,
# to check along the way that the CORS allowlist accepts it:
curl -s -o /dev/null -w '%{http_code}\n' -X POST \
  https://propnexus-api.onrender.com/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -H 'Origin: https://propnexus-web.onrender.com' \
  -d '{"email":"nobody@example.com","password":"wrong"}'      # 401
```

That last one is the canary and it is not optional. A `502` there means the app's most common error path is broken.

3 · Rollback

Render keeps previous deploys: **Dashboard** → **the service** → **Deploys** → **Rollback** to the last green one. It is instant and does not touch the database.

The database does not roll back with the service. If the problem was a migration, rolling back the code leaves the schema ahead. That is why migrations between deploys must be **additive**: a migration that only adds is compatible with the old code. If a migration ever has to be reverted, it is a new migration that undoes it — never editing the one already applied.

Turso free includes **1 day of point-in-time restore**:

```
turso db shell propnexus-west                                    # inspection
turso db create propnexus-restore --group propnexus --from-db propnexus-west --timestamp <ISO-8601>
```

Restoring creates a **new** database: point `DATABASE_URL` at it and redeploy. The original is not overwritten until you are sure. It needs `--group propnexus`: `--from-db` does not copy across groups.

Rolling back the region change (2026-10-06): `propnexus`, in Virginia, is the database from before the cutover and has received no writes since. Going back means pointing `DATABASE_URL` and one of its tokens at it, knowing it loses whatever was written in Oregon after the cutover.

4 · Incidents

Symptom	Most likely cause	What to do
First request takes ~1 min	Spin-down after 15 min. **Expected**	Nothing. Before a demo, warm it up by hand (§5)
The API does not start, the log mentions `JWT_SECRET`	The variable ended up empty	By design. Regenerate it in the dashboard and redeploy
Every login returns 429	`TRUST_PROXY_HOPS` other than 1	With 0 behind the proxy, every client shares one bucket. Set it to 1 and restart
Login `POST` returns **502**	The app's error path is broken	Check `apps/web/vite.config.ts` and **rebuild the web app** (it is build time)
The web app loads but every call fails	`VITE_API_ORIGIN` wrong or pointing to an old URL	Build time: fix the variable and **Clear build cache & deploy**
The web app loads and the console says CORS	The web app's origin is missing from the API's `WEB_ORIGIN`	Runtime: fix the variable and **restart** the API
Uploaded evidence disappeared	With R2 this should no longer happen	It is an incident. Check that `STORAGE_DRIVER=s3` is set: with `disk` it goes back to the ephemeral filesystem and is lost
The API does not start, the log says `STORAGE_DRIVER=s3 exige …`	An `S3_*` variable is missing	On purpose. Load the variable in the dashboard and restart (§1.4)
Service suspended mid-month	The 750 hours ran out	Someone set up a keep-warm. Remove it (§Accepted limitations)
A service stuck on an old commit, both `live` and no errors	The push arrived while the service was suspended	Resuming does not recover it. `render deploys create <srv-id>` (§2)

Symptom	Most likely cause	What to do
A push with green CI that touches the `buildFilter` and **no deploy** in `render deploys list`	Render did not create it. No pattern in the files: `dab082d` (2026-10-02) and `fb81ba9` (2026-10-07); in the same period, `9cce336` (three commits, the last one outside the filter) was created	`render deploys create <srv-id> --commit <sha> --wait`, the API before the web app (§2). Caught by checking each service's commit after every push, not only after resuming (§2)
Deploy in `update_in_progress` and `SIGTERM` to the old instance **and** the new one, ~15 min after the last request	The free tier puts the service to sleep after 15 min without requests, and that kills the instance being deployed	A request to `/health` (§2, "The deploy that coincides with the inactivity shutdown"). If it does not unstick it, wait for `update_failed` and redeploy
Screens that used to work start failing to parse	Front end and API on different commits	Same case as above. The `z.strictObject` schemas in `packages/shared` make it strict: a missing field breaks the whole parse
`ERR_PNPM_OUTDATED_LOCKFILE` in the build	A `package.json` was edited without `pnpm install`	CI catches it first; if it got here, run `pnpm install` and commit the lockfile
`Port scan timeout reached, no open ports detected` and then `Timed Out`	The build succeeded and the process **never listened**. The `startCommand` is `migrate && server`	Read the startup lines in order (below). Render takes ~15 min to declare it dead and on the free tier **the old instance is already gone**: it is an outage, not a degradation
`Error: Cannot find module '@some-otel-package'` right after an `instrumentation` log line	A late `require()` uses a package that is not in `dependencies` — it resolved as a transitive dependency locally, not in a clean install	Declare the package as a direct dependency; the CI smoke test (`.github/workflows/ci.yml`) already catches it before pushing
Deploy stuck in `update_in_progress` for 5+ min, log stops right after `[instrumentation] Sentry activo`	Seen once on 2026-09-08, cause undetermined — the old instance had already received `SIGTERM` and the API was really down	Cancel the stuck deploy (dashboard or `render deploys cancel`) and trigger a new one with `render deploys create <srv-id>`. The second attempt, with the same environment, started cleanly in ~1 minute; treated as a transient platform blip until it repeats

Reading a startup in the logs

The `startCommand` has two steps and each one announces its start and end. Read them in order — where the log stops tells you where the process hung:

```
[migrate] conectando a la base                      ← migrate started ("connecting to the database")
[migrate] sin migraciones pendientes                ← migrate finished ("no pending migrations", or "N applied")
[arranque] migraciones listas, levantando la API    ← server.js started ("migrations ready, starting the API")
API listening on http://localhost:10000             ← listening: from here Render can declare it live
AnchorPort listo en modo "real"                     ← the anchoring port resolved
```

Cold, on the free plan, a healthy startup reaches `API listening` in under a minute and `AnchorPort listo` one minute later (measured on 2026-10-07). If the second is missing, the database is not responding and a 120 s ceiling cuts it off with `la base no respondió en 120s`. If the first three are there and `API listening` is not, the problem is the server, not the migration.

This exists because of the 2026-09-04 incident, when none of these lines were printed: a startup hung in the migration and one hung in the server looked exactly the same — an empty log — and the API was down for ~18 minutes over a documentation-only commit. **Corrected on 2026-09-22:** that startup did not hang; Render shut it down for inactivity (§2, "The deploy that coincides with the inactivity shutdown"). The startup logs are still worth what they say: without them it could not have been seen.

Logs: **Dashboard → the service → Logs** (or `render logs -r <service>`). There is no shell: what is not logged cannot be inspected. That is why the API **fails at startup** instead of failing on a request.

Incident of 2026-09-07, two failures in a row on the same push. When instrumenting Sentry/OpenTelemetry (`instrumentation.ts`, preloaded with `node --require`), two consecutive deploys failed: first, `--require` without `./` (it resolves as a `node_modules` package, not as a file — `MODULE_NOT_FOUND` at startup); then, after fixing that, `NODE_ENV: production` declared as a service environment variable broke the **build** (`pnpm install` skips `devDependencies` with that variable set, and `packages/cardano` lost `@types/node`). The real fix was to make the `buildCommand` immune to what the environment brings (`NODE_ENV=development` inline before `pnpm install`), instead of depending on which variables the Blueprint declares. **The general lesson:** any change to `render.yaml` on an already-deployed service is reproduced locally with the **literal** command — build and start, in that order, with `NODE_ENV=production` exported by hand to simulate Render's environment — before pushing. A green `pnpm verify` does not prove the deploy will start.

5 · Before a demo, review or recording

Both URLs go to sleep. Warm them up by hand, ~2 minutes before:

```
curl -s -o /dev/null https://propnexus-api.onrender.com/health
curl -s -o /dev/null https://propnexus-web.onrender.com/
```

Do not automate this with a cron. A periodic keep-warm keeps both services awake 24/7 (~1,460 h against the plan's 750) and suspends them around day 15 — so the trick to avoid a one-minute cold start ends up causing a two-week outage. It is deliberately avoided, not forgotten. A manual, attended loop that lasts as long as a recording and is stopped at the end is a different thing and is fine.

5.1 · Reverting the data preparation of 2026-09-21

On 2026-09-21 the database was prepared to record the video walkthrough. Three changes, all on demo data and none on the schema:

1. **`buyer@example.com` added as a member** of `torre-volumen-1` and `torre-volumen-2`, so its listing no longer shows only two projects.
2. **The three `torre-volumen-*` projects moved from `planning` to `completed`**, which matches their 10 certified stages.
3. **`torre-a` made unreachable from the front end, without leaving the database.** It has 3 stages named before the 10-stage default catalog and the only `Failed` on-chain event in the whole database, so it was confusing on camera. **It was not deleted: it has 8 on-chain events attached.** Its 3 memberships were removed — membership is the real visibility mechanism —, its unit `4B` was unlinked (*My units* is listed by `Unit.investorId`, not by membership), and its `Dossier` row was deleted, since otherwise it kept appearing in the notary's queue.

Rollback of point 3, the only one that loses information if not written down:

```
INSERT INTO ProjectMember (id, userId, projectId, membershipRole, createdAt) VALUES
  ('d703n50ggfir59g0wnl2hf8a','hnrykorp4aqul78oiy9bfe4h','m99yzb4h5poi0078rcqpbj6d','developer',1788447379597),
  ('s6iuiow6lsyi6h57itt7m8k3','ng0gh91de5alybr5ihupbd11','m99yzb4h5poi0078rcqpbj6d','buyer',1788447379598),
  ('ejsq8ei1ufnuz1uk0ura1564','k2knwiqp66xuv6ojp7iv48ep','m99yzb4h5poi0078rcqpbj6d','verifier',1788447379599);

UPDATE Unit SET investorId = 'ng0gh91de5alybr5ihupbd11', status = 'sold'
  WHERE id = 'f3qugedzwjnzhkwth5kqwf3n';

-- `compileDossier` regenerates this row on the next read, but with another id.
-- This INSERT keeps the original id and masterHash.
INSERT INTO Dossier (id, unitId, masterHash, compiledAt, shareToken, status, signedById, signedAt, rejectionNote)
       VALUES
  ('wjji7ls3xm5nro9ehxxur7tp','f3qugedzwjnzhkwth5kqwf3n',
   '8426e9e08fae94ce5de2c38fd0cf8c95ee1f0c181db77f48447dab09de5f8a3e',
   1788447381415, NULL, 'compiled', NULL, NULL, NULL);
```

Applied with `turso db shell propnexus-west < file.sql`. **This is not a migration and does not belong in `apps/api/migrations/`**: there it would apply itself, on every startup, against any database.

Points 1 and 2 are reverted as admin: the two new memberships have no delete endpoint (done with `turso db shell`), and the status goes back with `PATCH /api/v1/projects/:id {"status":"planning"}`.

Accepted limitations (read before promising anything)

1. **~1 min cold start** after 15 min of inactivity. Accepted in exchange for $0.
2. **Evidence persists on R2** (§1.4), not on the filesystem: an anchored hash always has a file behind it. The filesystem is still ephemeral, and that is fine — `UPLOAD_DIR` is only the upload staging area and the route deletes the temporary file as soon as R2 confirms. What needs watching is the free tier's **10 GB** ceiling.
3. **750 instance-hours/month shared** by both services. With normal spin-down it is plenty (~1,500 cold visits); with a keep-warm it is not.
4. **No confirmation worker.** Render background workers have no free tier: reconciliation is triggered by a GitHub Actions cron against an authenticated endpoint (`reconcile.yml`, daily) and on read — **never a `setInterval` inside the API**, which stops counting when the service sleeps.
5. **Monitoring: live since 2026-09-08.** Sentry (errors, back end and front end), OpenTelemetry → Grafana Cloud (back-end traces) and PostHog (web analytics, no personal data) are instrumented (`apps/api/src/instrumentation.ts`,

`apps/web/src/lib/observability.ts`) and verified with real production data (screenshots). Without `SENTRY_DSN` / `OTEL_EXPORTER_OTLP_ENDPOINT` they stay off without breaking anything. Render Metrics Stream (native infrastructure metrics: container CPU/RAM) is a Pro+ feature, not free — Render's free dashboard is enough to look at them, they just cannot be exported to Grafana Cloud without upgrading.